IntoToArduino_Student.odt
17dec25
09may26

Introduction to Using the Arduino Development System

ref1: https://docs.arduino.cc/hardware/uno-rev3/

Student Notes

Table of Contents

0. Setting up the Arduino IDE

Go to the Arduino website at

https://www.arduino.cc/ > For Makers > Go to Documentation > Software > Arduino IDE 2

There is a walk-through for installing the Arduino IDE and hardware plus lots of other valuable information about the Arduino product line, project ideas, and additional help.

1. Blink; the hardware “Hello World” application.

This program can be found in

File > Examples > 01.Basics > Blink

Click Blink to load the program into the IDE. Connect your Arduino board to the computer using the USB cable. In the pull-down “Select Board” in the menu bar, select the board connected if the system has not already set this for you.

This program is stored in the installation folders and these should not be changed. So the first thing to do, if you want change this program is to save into your own folder. Use File > Save As… and select the path for your folder. In the folder, unclick the read-only file property of the Blink.ino file and load the file from your folder into the IDE for editing. I use “C:\Users\<your name>\Documents\Arduino” for my files.

On line 34, change the delay(1000) to delay(100). Click the Upload arrow and the LED should now Blink quickly each second. This shows that you have control of the hardware though your programming.

2. LEDs

Some electronics theory is now needed so that parts (the Arduino or the LEDs) are not damaged.

The first question to be answered is “What does it take to light up an LED?”. The standard red LED requires between 1.8v and 2.2v across it and at least 1ma of current available. The schematic symbol and physical case are shown in Figure 1. The FLAT side of the circular LED case indicates the Cathode or negative terminal.

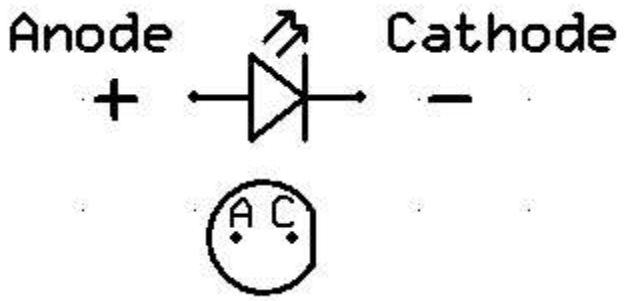

Figure 1

The Anode is connected to the positive voltage source and the Cathode is connected to the negative or ground side. Current (by convention) flows from positive to negative.

At 1ma., it will be very dim, but will light up. Most LEDs can handle 20ma and up to 30ma for a short duration. We will assume that the LED drops 2v and will use 10ma for brightness. Brightness can be controlled by how much current is used, but the voltage drop across the LED will not change.

Looking at the Arduino UNO 3 hardware specifications (see ref1 Tech Specs), the Arduino I/O pins can supply 5V @ 20ma.

The problem then is provide the correct voltage to the LED at a desired current. The solution is to use a resistor in series with the LED and Ohm's Law to calculate the resistor's value.

This is a simple circuit for the Arduino connected to an LED.

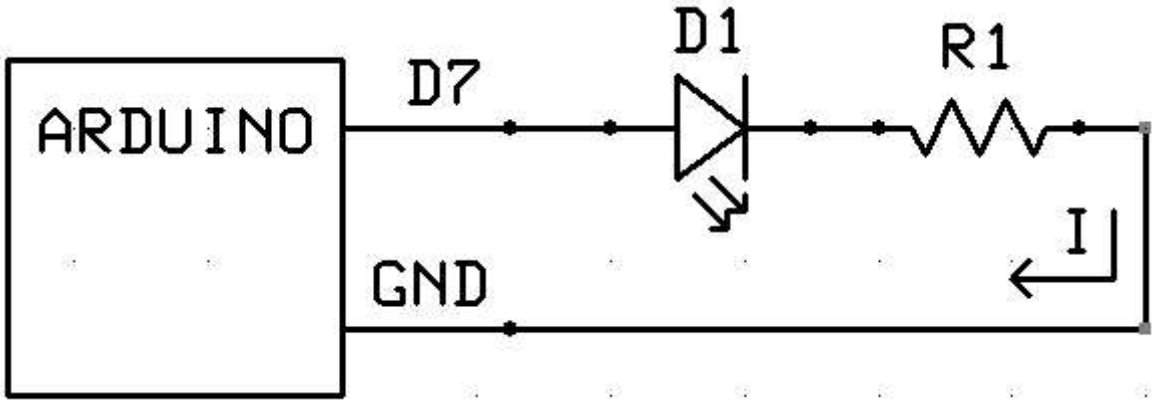

Figure 2

It is a 'series' circuit because all of the components are strung together in a series, one after another. There's another law called Kirchoff's Voltage Law (KVL) that states that "The sum of the voltages (drops and rises) in a series circuit must add up to zero." Another Kirchoff Law states the "Current entering a node equals the current exiting a node." In a series circuit, each connection is a node with one input and one output. Therefore all of the connection have the same current (I) flowing though it and each component has the same current flowing though it. (i.e. the current is the same everywhere in the circuit).

Following the circuit in Figure 2, the voltage starts a +5v (Arduino I/O Pin), then drops 2v across the LED, then drops Vr across the resistor, and ends up at 0v and Ground (GND). Using KVL, 5 – 2 – 3 = 0, so the resistor must drop 3 volts. Using Ohm's Law that E = IR or Voltage = Current x Resistance, the known values are plugged into the equation. V_r = I x R. V_r and I are know. R is to be solved for. Rearrange the equation to solve for R.

$V_r/I = R \rightarrow$ 3v / 10ma = 3v / 0.01A = 300 ohms.

The nice thing about LEDs is that their brightness is subjective; a little dimmer or a little brighter is hardly noticeable. If a 300 ohm resistor is not available, use a 270 ohm (11ma) or a 330 ohm (9ma) resistor. These are common values for resistors. Other common values are 150, 220, 390, and 470. Any of these will cause the LED to light up.

3. Adding external LEDs to the Arduino aboard.

Apply information from #2 to preform experiments with six LEDs. Connect up to six LEDs to the Arduino board as shown in Figure 3. Different resistor values may be used based on availability or desired brightness.

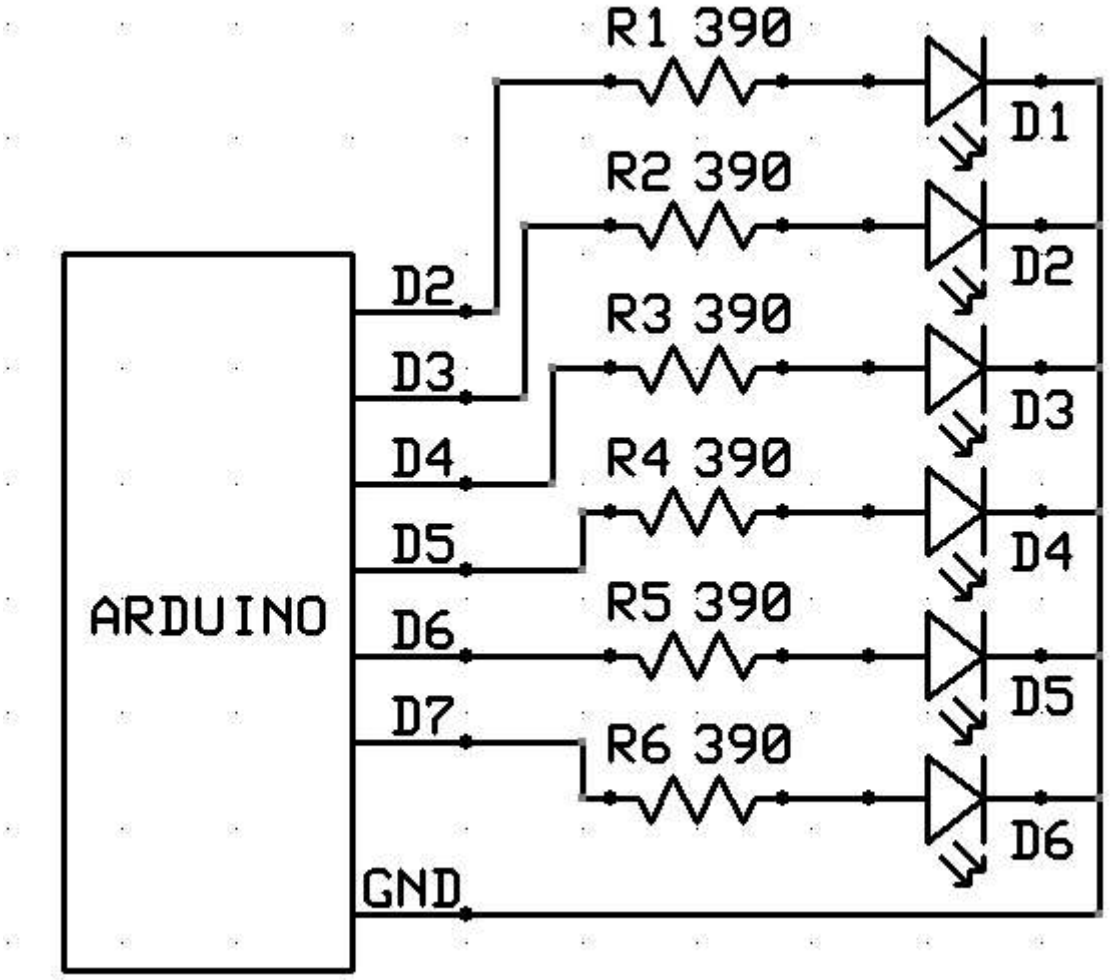

Figure 3

3a. Turn each LED ON; one at a time, then turn each LED OFF, one at a time and repeat.
In programming, there are many ways to do the same thing. The follow section will explore this concept.

First though, a few words on how the Arduino processes code. Simple projects can be written on 'C'. C++ and Python are also available.
The Arduino starts up by running it's Bootloader. This block of code contains the support functions, timing, configuration code, etc. Once the Bootloader completes initialization, it calls the setup() function in the User sketch (code). The setup() function is expected to return back to the Bootloader. Any value returned is ignored. The Bootloader will then call the User loop() function, which is expected to return back to the Bootloader in a reasonable amount of time, say less than one second. It then calls loop() again. The call/return process continues until the User sketch decides to stop.
In summary, setup() runs once at the beginning of the sketch and loop() is called over and over again.

3a1. Straight line code.
It's usually easiest to start with a straight-line code approach to any new problem. This can provide insight into how the hardware controls an external device and logical issues of control.
Using the Blink sketch as a reference, a few things need to be added to control more LEDs connected to the I/O pins.

Use File > Open > the LED_3a1 file, located in the ITA_Examples folder provided, to load the example code into the IDE.

Reading though the sketch, the first section gives a name to each 'magic' number used by the hardware control and configuration functions. LED2 is more informative than 3.
The next section sets up the I/O lines to drive the LEDs. Look up these functions in the Reference for more information.
The last section is the loop() function. Each time it is called, it steps through the process of turning ON/OFF each LED using a 100ms delay between each change.

Possible Modifications
1. Change the sketch to turn each LED, with a small delay between each, and then turn each LED OFF, with a small delay between each.
2. Change the sequence order of which LED are turned ON or OFF.

3a2. Using an **Array** to control which LED is Blinked.
Use File > Open > the LED_3a2 file.

This sketch results in the same display as the 3a1 sketch, but in a different manner.
It places all of the LED references into an array and then steps though the array extracting the LED references one at a time using a for() loop control structure.
Using a for() loop for a process that does the same operation for different values can reduce errors or cause then to show up earlier since the same code is used for all values.

The array has been added to the first section.
The setup() code is reduced to a for() loop the walks through the array to set up the I/O.
The loop() code is also reduced. It's process has been changed to do one LED blink per loop() call.
The index variable is 'static' so that it retains its value between loop() calls. This variable could have been placed above the setup() to make it a global variable which would give the same results.
(see Help > Reference > Variable Scope & Qualifiers for more information)

Possible Modifications
1. Add more LEDs to the array (from the available LEDs) to increase the display.
2. Change the delay() times to make the display cycle faster or slower.

3a3. Using an Array of **Data Structures** {LED#, State, time} to control output and delay.

Use the Array as a circular pattern generator.
Use the Array as a scrolling patter generator; Chase, Ping-pong, Morse Code.

3a4. Using the random function to randomly pick an LED from the Array to turn ON/OFF or Toggle.
Use an Array to record the **State** of an LED.

3b. Use a **State Machine** to cycle through different display types.
3b1. Use time checks to set the run-time for each state.
3b2. Use a **Table** to control the state sequence.

4. 7 Segment LEDs

4a. Single 7-Segment LED Display

4a1. Using Arduino functions – sketch LED7seg_4a1.ino

A 7-segment LED is just 8 LEDs arranged to display characters. The anode of the LED is attached to it's respective letter pin (a, b, c, etc.) and all LED cathodes are attached to both of the C (common) pins 3 and 8. So, the connections are the same as in Section3, except for eight LED instead of 6. The other difference in the hardware is the use of a transistor to sink the current from all of the LEDs. An Arduino pin can only sink 20ma. If all of the LEDs are ON, then the common line could have as much as 160ma which is beyond the Arduino's capability. Therefore, a NPN transistor is used.

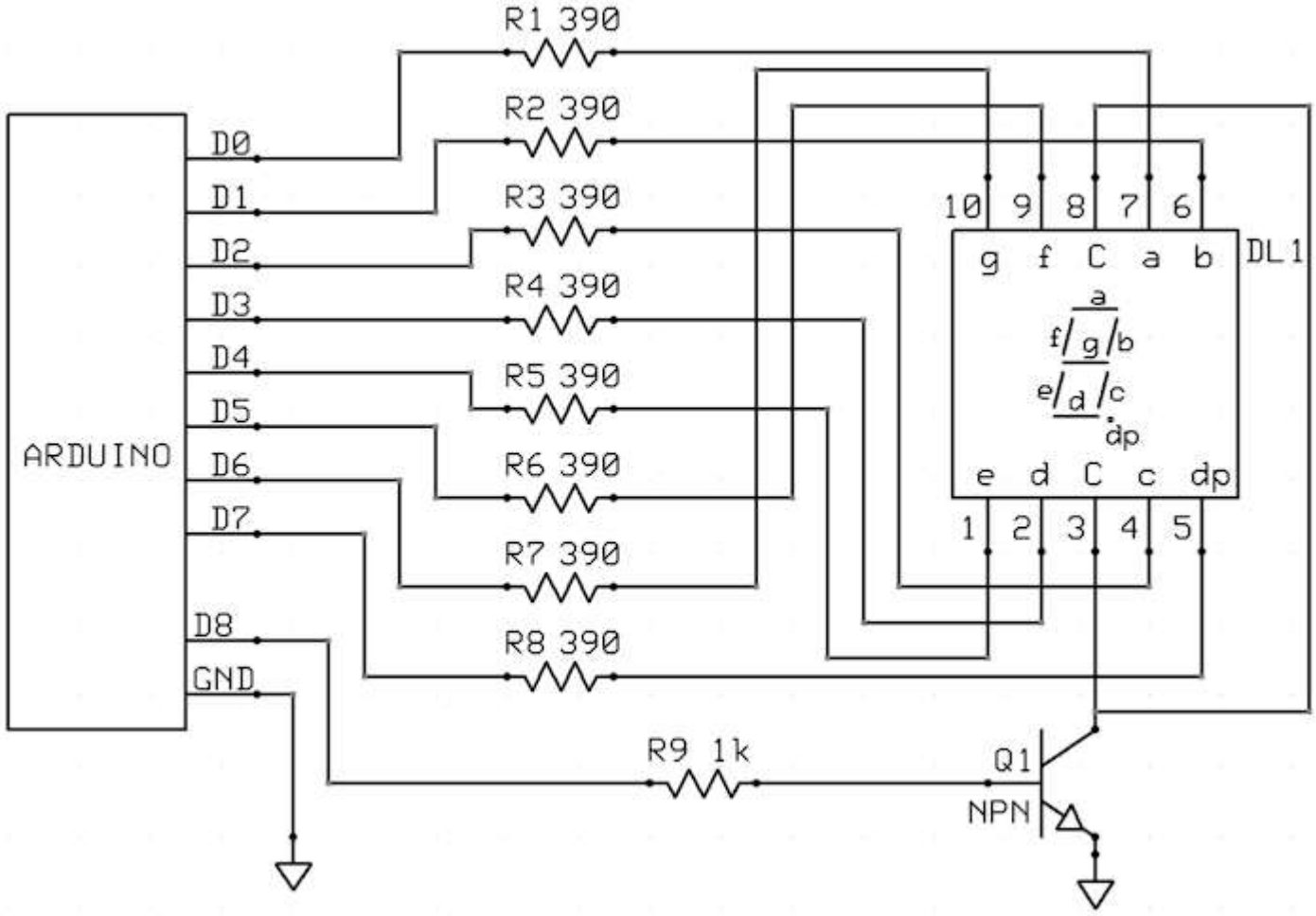

Figure 4

4a1a. Removing Delay() calls.

4a2. Using direct hardware control registers

4b. Multiple 7-Segment LED Displays